Pensamento Computacional
IN05008

Aula 5 – Expressões Condicionais

Prof. Lucas Nunes Alegre

Universidade Federal do Rio Grande do Sul

Instituto de Informática

Departamento de Informática Teórica

Slides baseados no material da Prof. Leila Ribeiro.

Expressões Condicionais

Em diversas situações, a ação a ser realizada depende de alguma condição:

- Se estiver chovendo, leve o guarda-chuva
- Se o sinal estiver vermelho, pare
- Se o bolo estiver pronto, retire do forno
- ...

Expressões Condicionais

Em diversas situações, a ação a ser realizada depende de alguma condição:

- Se estiver chovendo, leve o guarda-chuva
- Se o sinal estiver vermelho, pare
- Se o bolo estiver pronto, retire do forno
- ...

Condições

O que é uma condição?

Definição. Uma condição é uma expressão cujo valor é:

- Verdadeiro; ou
- Falso

O conjunto Boolean contém apenas os dois valores booleanos:

- Boolean = {verdadeiro, falso}

Em Pyret:

- Boolean = {true, false}

Expressões Booleanas

- 1 < 2 true
- 3 == 4 false
- 3 > 4 false
- x > 1 ???

Expressões Booleanas

- 1 < 2 true
- 3 == 4 false
- 3 > 4 false
- x > 1 ???

< : Number Number → Boolean

== : Number Number → Boolean

> : Number Number → Boolean

- Em Pyret (e muitas linguagens de programação), usamos "==" no lugar de "=" para comparações.

Operadores Genéricos e Específicos

Podemos usar == para comparar números, strings, etc.

- "a" == 1 false

Se quisermos comparar valores de um tipo específico, podemos usar operadores específicos:

- num-equal(1, 1) true
- num-equal(1, 2) false
- string-equal("a", "a") true
- string-equal("a", "b") false

P. Tente executar em Pyret num-equal(“a”, 1). O que acontece? Por quê?

Operações Booleanas

- not : Boolean → Boolean
- and : Boolean Boolean → Boolean
- or : Boolean Boolean → Boolean

- not(true) == false
- not(false) == true

- true and true == true
- true and false == false
- false and true == false
- false and false == false

- true or true == true
- true or false == true
- false or true == true
- false or false == false

Exemplos

- Se tem sol eu vou na praia.

  C1

- Se não tem sol eu vou na praça.

  not(C1)

- Se não tem sol e tem pipoca eu vou assistir TV.

  not(C1) and C2

- Se tem chocolate ou tem pipoca eu vou assistir TV.

  C3 or C2

Exemplos - Pyret

- (1 == 2) and (2 < 3) — false
- (1 == 2) or (2 < 3) — true
- not(2 < 3) — false
- not(x < 3) — ???
- (x == y) and (x < y) — false
- (x == y) or (x < y) or (x > y) — true

Erro de execução caso x e y não forem definidos.

Exercícios

Exercício 1. Qual o resultado das seguintes expressões em Pyret?

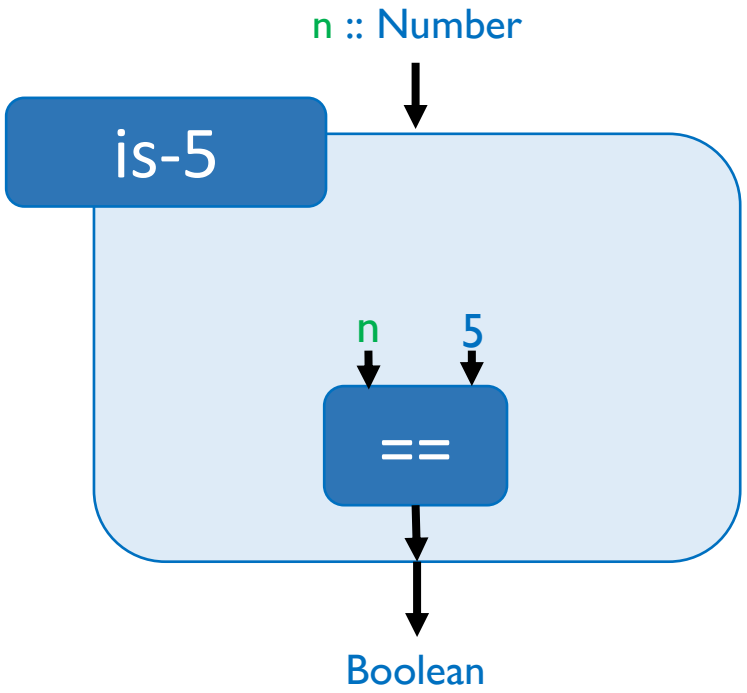
1. (4 > 3) and (10 <= 100)
2. (4 > 3) or (10 <= 100)
3. not(2 == 3)

Exercício 2. Qual o resultado de:

1. (4 > 3)
2. (4 > x) and (x > 3)
3. (x * x) == x

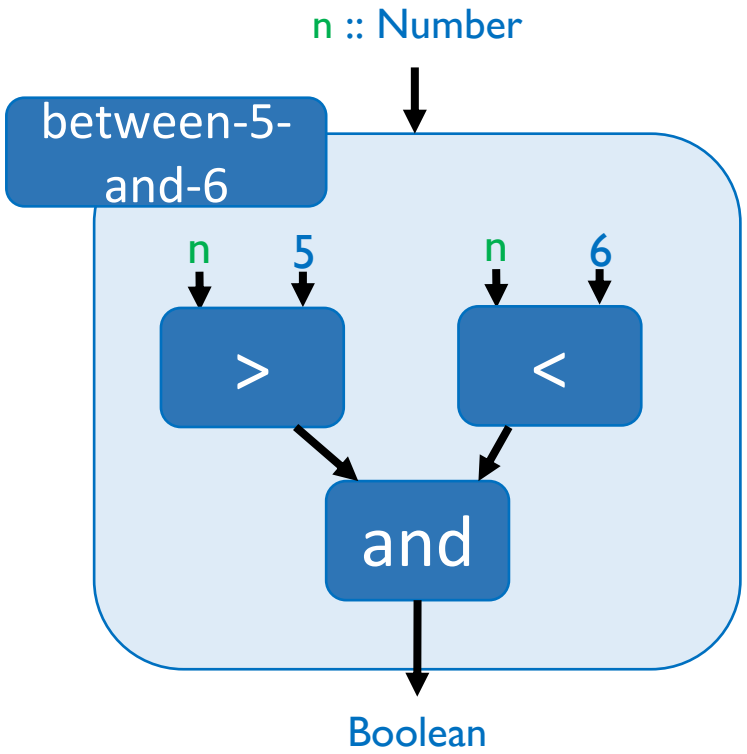
Para (a) x = 4; (b) x = 2; e (c) x = 7/2.

Exemplo



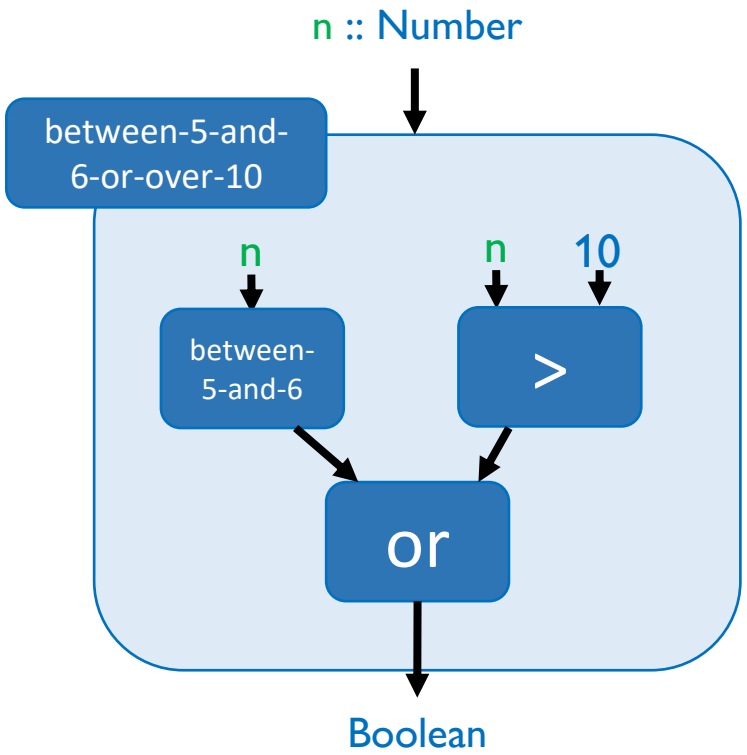

```
fun is-5(n :: Number) -> Boolean:
  doc: "Dado um número verifica se ele é 5."
  n == 5
where:
  is-5(5) is true
  is-5(2) is false
end
```

Exemplo




```
fun between-5-and-6(n :: Number) -> Boolean:
  doc: "Dado um número verifica se ele está
        entre 5 e 6."
  (n > 5) and (n < 6)
where:
  between-5-and-6(5.2) is true
  between-5-and-6(5) is false
  between-5-and-6(0) is false
end
```

Exemplo




```
fun between-5-and-6-or-over-10(n :: Number) -> Boolean:
  doc: "Dado um número verifica se ele está entre 5 e 6
   ou é maior que 10."
  between-5-and-6(n) or (n > 10)
where:
  between-5-and-6-or-over-10(5.5) is true
  between-5-and-6-or-over-10(11) is true
  between-5-and-6-or-over-10(7) is false
end
```

Exercício

Exercício. Traduza os intervalos abaixo para funções em Pyret que aceitem um número e produzem true se o número está no intervalo e false, caso contrário.

1. (3, 7]
2. [3, 7]
3. [3, 9)
4. (1, 6) ∪ (9, 14)
5. Na parte de fora de [1, 3]

Exercício

Exercício. Traduza as funções abaixo para intervalos:

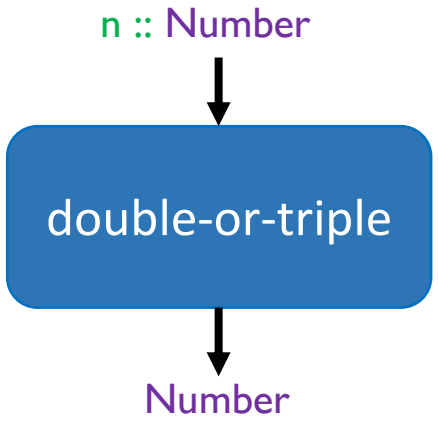
```
fun interval-1(n :: Number) -> Boolean:
  (-3 < n) and (n < 0)
end


fun interval-2(n :: Number) -> Boolean:
  (n < 1) or (n > 2)
end


fun interval-3(n :: Number) -> Boolean:
  not((1 <= n) and (n <= 5))
end
```

Exemplo

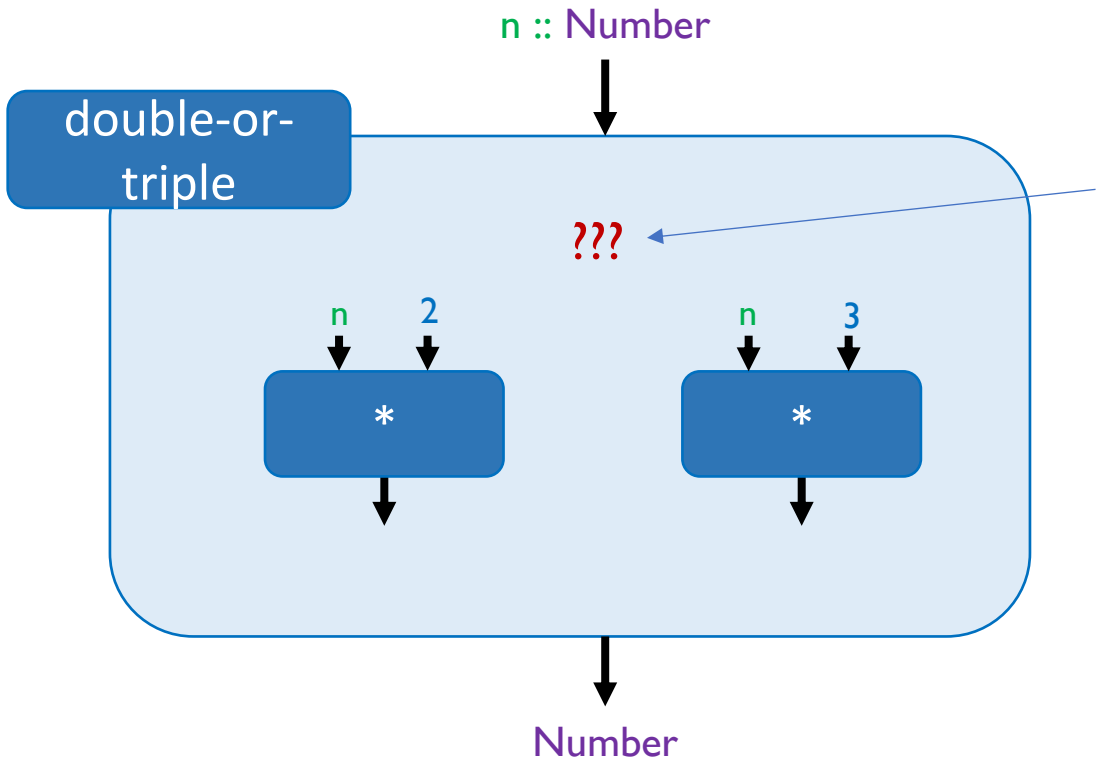
Vamos construir um algoritmo que, dado um número, se ele for maior que 10 devolve o seu dobro, senão, devolve o seu triplo.



double-or-triple :: Number → Number

Exemplo

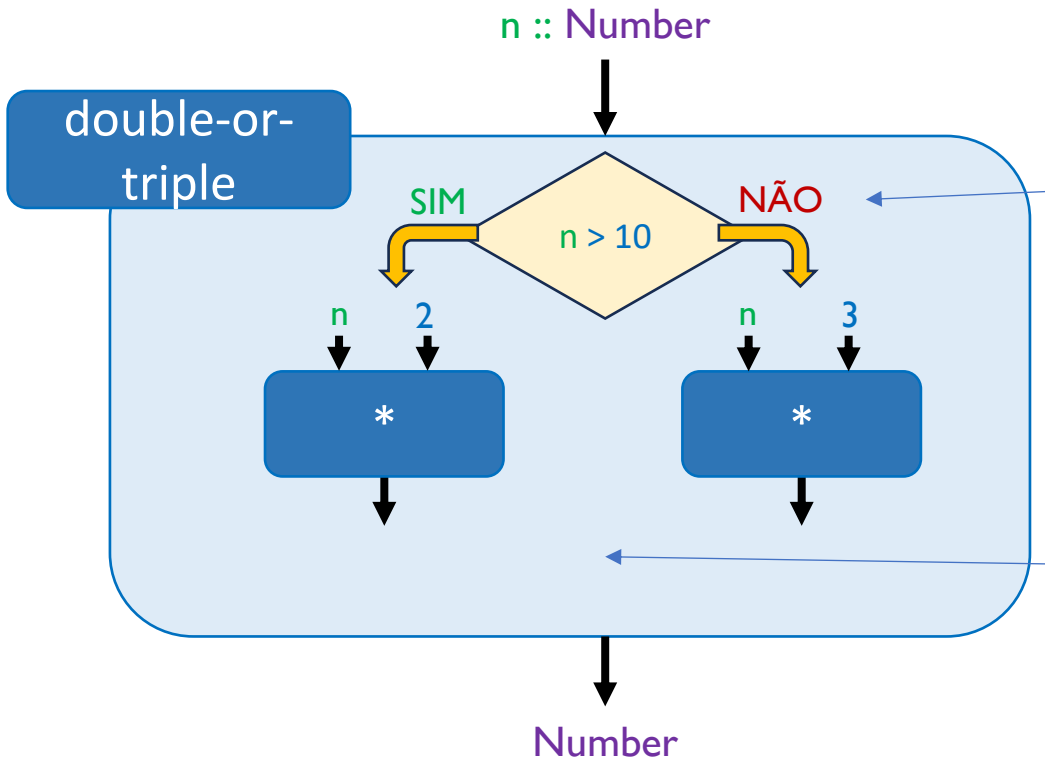
Vamos construir um algoritmo que, dado um número, se ele for maior que 10 devolve o seu dobro, senão, devolve o seu triplo.



Como escolher qual ação deve ser realizada?

Exemplo

Vamos construir um algoritmo que, dado um número, se ele for maior que 10 devolve o seu dobro, senão, devolve o seu triplo.

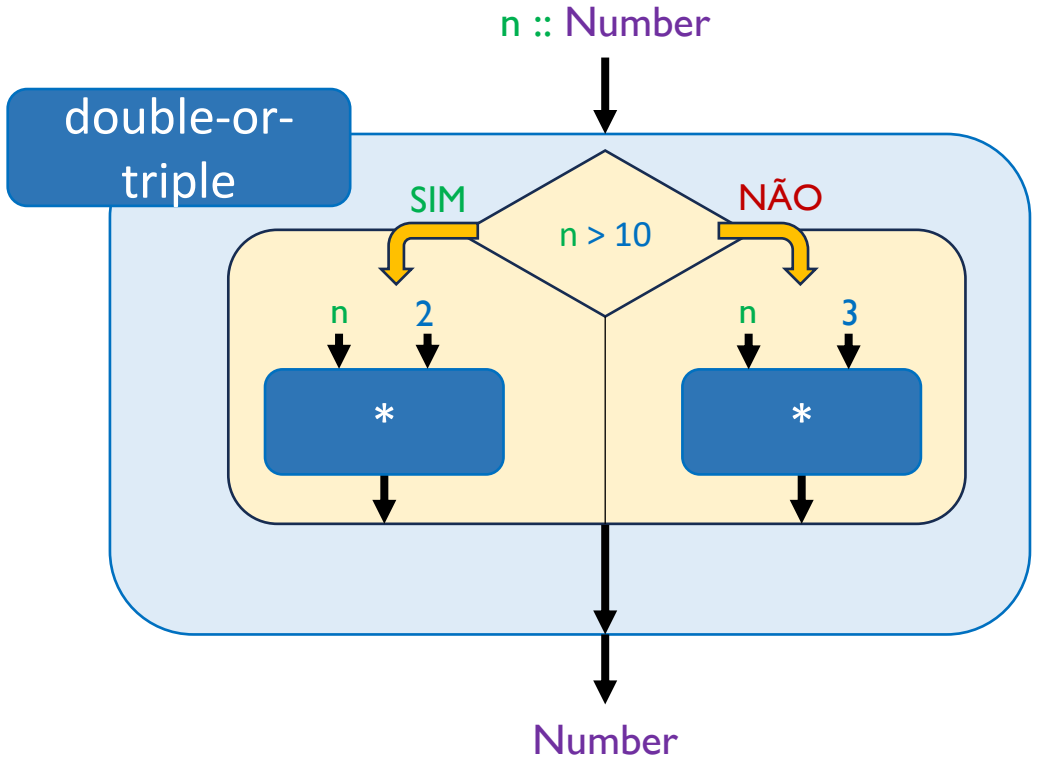


Fazendo um teste!

Como fazemos para termos apenas um resultado?

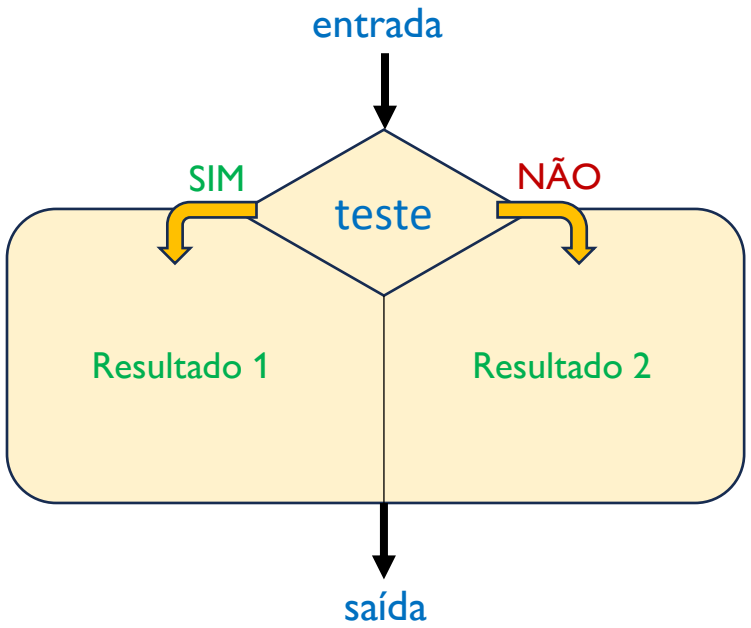
Exemplo

Vamos construir um algoritmo que, dado um número, se ele for maior que 10 devolve o seu dobro, senão, devolve o seu triplo.



Encapsulamos os resultados em um **bloco condicional**!

Expressão **if**



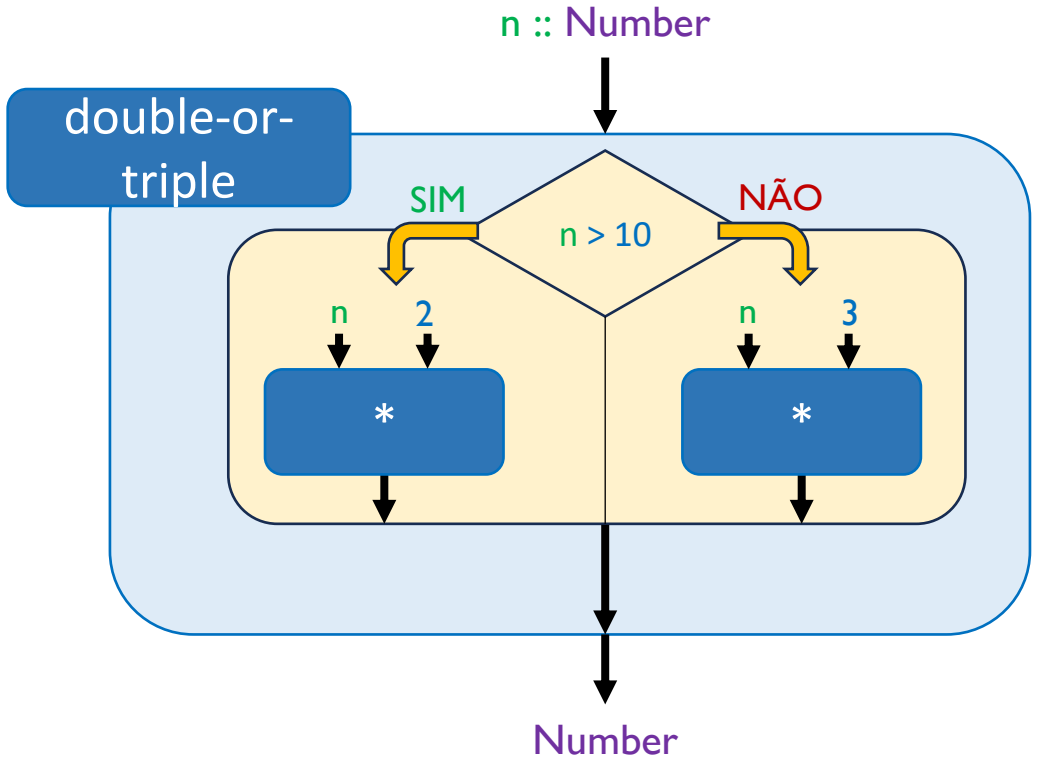
Pyret:

```
if <test>:
  <resultado1>
else:
  <resultado2>
end
```

- O teste deve ser uma expressão booleana
- O resultado deve ser uma expressão válida (qualquer tipo)
- Nunca mais de uma instrução de uma condicional é executada

Exemplo

Vamos construir um algoritmo que, dado um número, se ele for maior que 10 devolve o seu dobro, senão, devolve o seu triplo.




```
fun double-or-triple(n :: Number) -> Number:
  doc: "Dado um número retorna o dobro se for
    maior que 10 e o triplo caso contrário."
  if (n > 10):
    n * 2
  else:
    n * 3
  end
where:
  double-or-triple(12) is 12 * 2
  double-or-triple(4) is 4 * 3
end
```

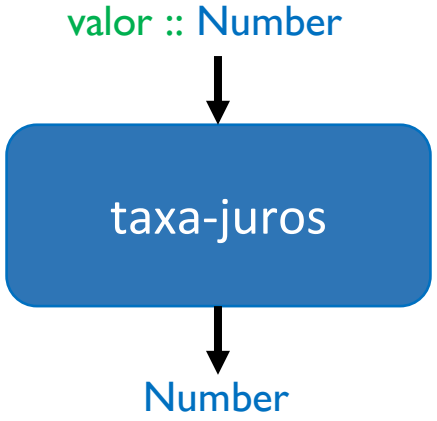
if: Funcional vs. Imperativo

- Em Pyret (e outras linguagens funcionais), o **if** é uma expressão!
  - É avaliada, e retorna um único valor.

- Em C (e outras linguagens imperativas), o **if** é um comando de controle de fluxo!
  - Não é avaliada para um valor, apenas controla que linhas serão executadas

Condicionais com múltiplos testes

Problema. Suponha que um banco pague juros de 4% para depósitos de até R$ 1000 (inclusive), 4.5% para depósitos de até R$ 5000 (inclusive) e de 5% para depósitos de mais de R$ 5000. Escreva um programa que, dado o valor a ser depositado, produza a taxa de juros correspondente a esse valor.



taxa-juros :: Number → Number

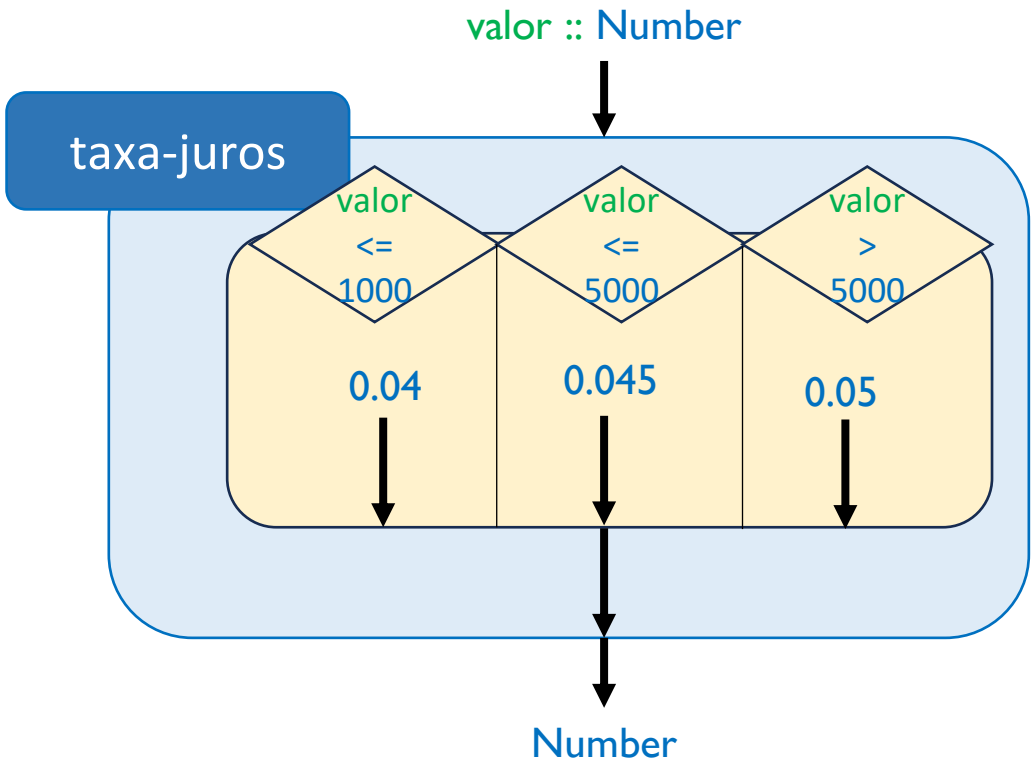
```
examples:
  taxa-juros(800) is 0.04
  taxa-juros(3000) is 0.045
  taxa-juros(6000) is 0.05
end
```

Encadeamentos de expressões **if**

- Podemos encadear diversas expressões **if-else**
- Se mais de uma expressão for verdadeira, a primeira será executada.
- Nunca mais de uma instrução de uma condicional é executada.

```
if <test1>:
  <result1>
else if <test2>:
  <result2>
  ...
else if <testN>:
  <resultN>
else:
  <resultDefault>
end
```

Condicionais com múltiplos testes




```
fun taxa-juros(valor :: Number) -> Number:
  doc: "Dado um valor de depósito, retorna
    a taxa de juros correspondente."
  if (valor <= 1000):
    0.04
  else if (valor <= 5000):
    0.045
  else:
    0.05
  end
where:
  taxa-juros(800) is 0.04
  taxa-juros(3000) is 0.045
  taxa-juros(6000) is 0.05
end
```

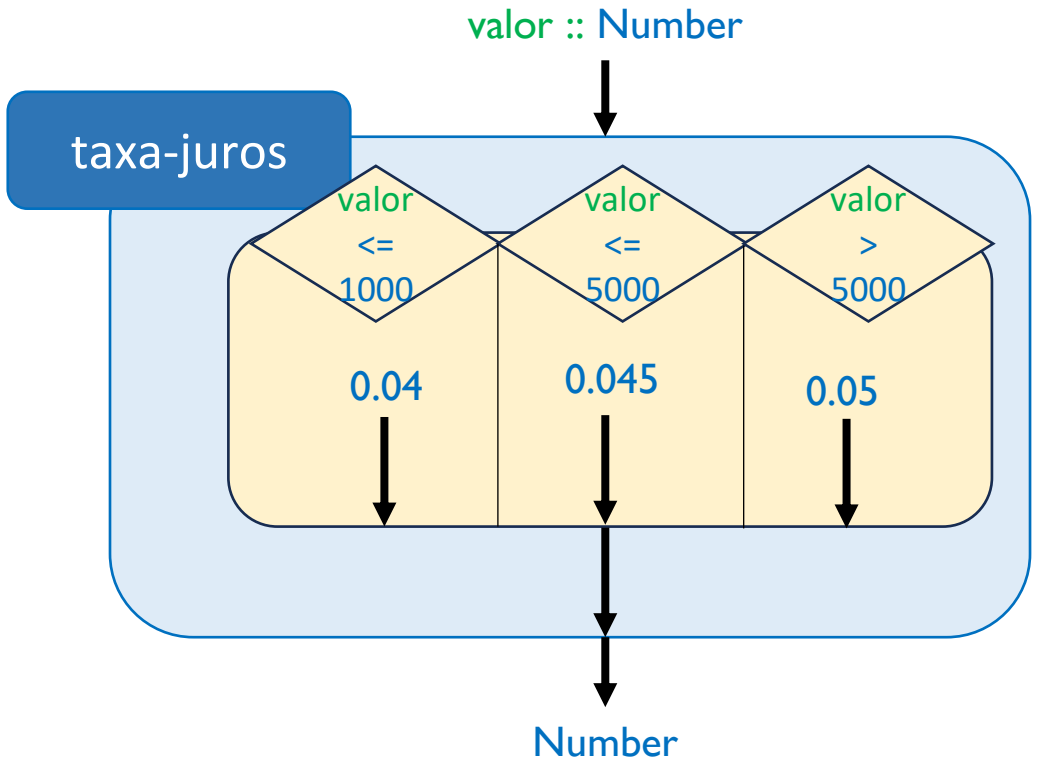
Expressão **ask**

- Em linguagens de funcionais, podemos escrever condicionais de outras formas.
- Em Pyret, podemos utilizar a expressão **ask:**

```
ask:
| <test1> then: <result1>
| <test2> then: <result2>
...
| <testN> then: <resultN>
| otherwise: <default-result>
end
```

- Nesta disciplina, iremos sempre preferir a expressão **ask** em lugar do **if** .

Condicionais com múltiplos testes




```
fun taxa-juros2(valor :: Number) -> Number:
  doc: "Dado um valor de depósito, retorna
a taxa de juros correspondente."
  ask:
  | (valor <= 1000) then: 0.04
  | (valor <= 5000) then: 0.045
  | otherwise: 0.05
  end
where:
  taxa-juros2(800) is 0.04
  taxa-juros2(3000) is 0.045
  taxa-juros2(6000) is 0.05
end
```

Avaliação da Expressão **ask**

```
taxa-juros2(6000)
  ask:
  | (valor <= 1000) then: 0.04
  | (valor <= 5000) then: 0.045
  | otherwise: 0.05
  end
  ask:
  | (6000 <= 1000) then: 0.04
  | (6000 <= 5000) then: 0.045
  | otherwise: 0.05
  end
  ask:
  | false then: 0.04
  | (6000 <= 5000) then: 0.045
  | otherwise: 0.05
  end
```

```
ask:
| (6000 <= 5000) then: 0.045
| otherwise: 0.05
end
ask:
| false then: 0.045
| otherwise: 0.05
end
ask:
| otherwise: 0.05
end

>> 0.05
```

Qual a diferença?

```
fun taxa-juros2(valor :: Number) -> Number:
  ask:
  | (valor <= 1000) then: 0.04
  | (valor <= 5000) then: 0.045
  | otherwise: 0.05
  end
end
```

```
fun taxa-juros3(valor :: Number) -> Number:
  ask:
  | (valor <= 1000) then: 0.04
  | (valor > 1000) and (valor <= 5000) then: 0.045
  | (valor > 5000) then: 0.05
  end
end
```

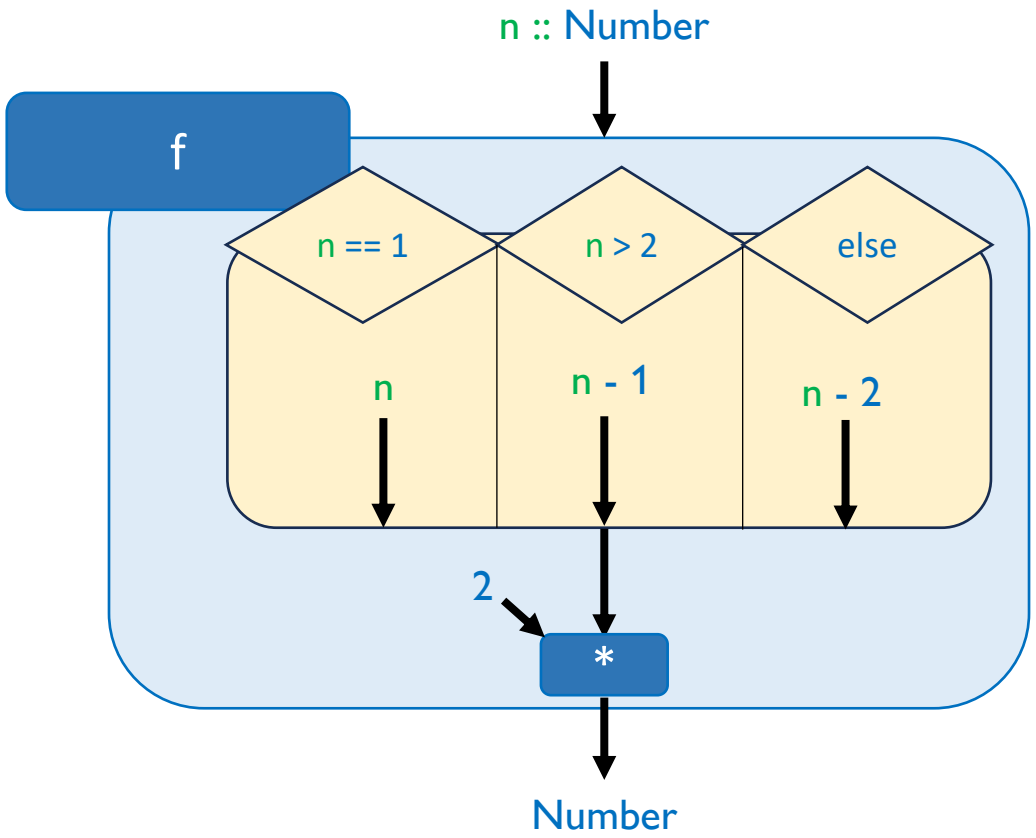
Exemplo

- f :: Number → Number

- $$f(n) = \begin{cases} 2 \times n, & se\ n = 1 \\ 2 \times (n - 1), & se\ n > 2 \\ 2 \times (n - 2), & caso\ contrário \end{cases}$$

- $$f(n) = 2 \times \begin{cases} n, & se\ n = 1 \\ (n - 1), & se\ n > 2 \\ (n - 2), & caso\ contrário \end{cases}$$

Exemplo



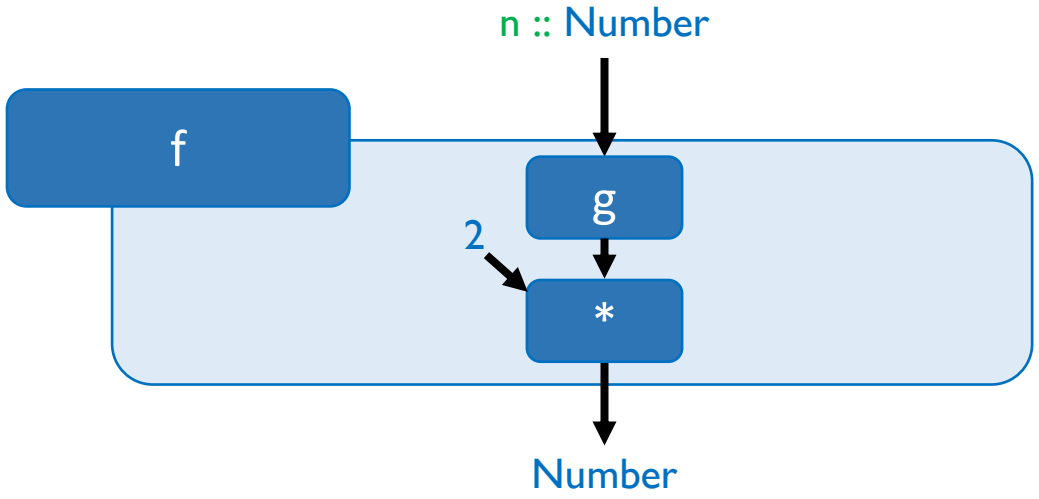
$$f(n) = \begin{cases} 2 \times n, & se\ n = 1 \\ 2 \times (n - 1), & se\ n > 2 \\ 2 \times (n - 2), & caso\ contrário \end{cases}$$

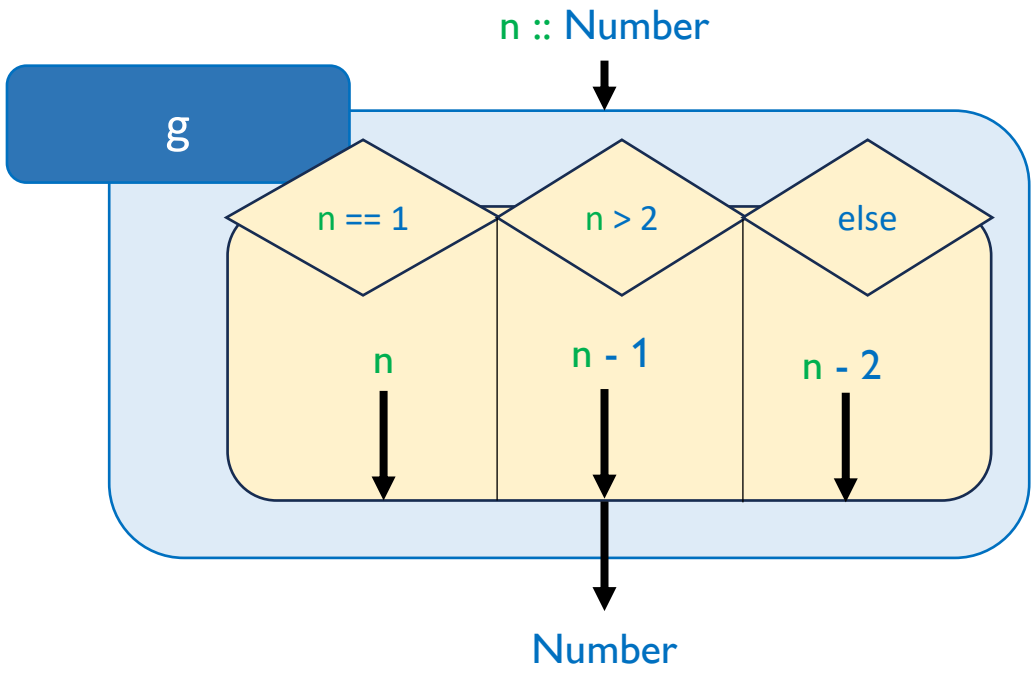
```
fun f(n :: Number) -> Number:
  2 * (
    ask:
    | n == 1 then: n
    | n > 2 then: n - 1
    | otherwise: n - 2
    end
  )
end
```

Exemplo

- f :: Number → Number
- $f(n) = 2 \times g(n)$
- $$g(n) = \begin{cases} n, & se\ n = 1 \\ (n - 1), & se\ n > 2 \\ (n - 2), & caso\ contrário \end{cases}$$

Exemplo






```
fun f(n :: Number) -> Number:
  2 * g(n)
end


fun g(n :: Number) -> Number:
  ask:
    | n == 1 then: n
    | n > 2 then: n - 1
    | otherwise: n - 2
  end
end
```

Lembrete

- Ler o capítulo 3 do livro.

- Fazer os exercícios em:
- https://lucasalegre.github.io/pensamento-computacional/exercises

